

Clearspace

30-Minute CS 499 Code Review Runbook

A step-by-step recording script: exactly what to show, what to say, and when to move on.

IMPORTANT

You are not expected to talk for 30 minutes without stopping. The time comes from showing the working app, navigating to each file, scrolling to the named lines, and explaining what the viewer sees. Read each “Then say” section at a calm pace, follow the listed “Pace” instruction, and then read the exact “Next say” sentence before you move to the following step.

Before you press Record

Turn to the next page and prepare every item in the checklist. Once the preparation is complete, follow the time-stamped script without needing to search for files.

Pre-recording checklist

Set this up before starting your screen recording. The goal is to make the recording calm and linear: the next screen or source file is already waiting when the script tells you to show it.

Prepare	Have this ready before Record
Recording screen	Clearspace open on a safe, ordinary demo folder with several files or folders visible. Do not use a folder that reveals private names, school records, or other personal information.
App demonstration	Make sure the sidebar and search box are visible. If possible, choose one file with an existing tag. If no tag is available, the script tells you what to say instead.
Project	Open the Clearspace project folder in your code editor. Keep the editor zoom comfortable enough for the viewer to read while you point at a line.
Code tabs 1-4	Open: Clearspace/ARCHITECTURE.md; Clearspace/ViewModels/MainViewModel.cs near RunTreeSearchAsync around line 1085; Clearspace/Commands; Clearspace/Services/FileOperationService.cs.
Code tabs 5-7	Open: Clearspace/Services/FileIndex.cs; Clearspace/Services/FileIndexBuilder.cs; Clearspace/Services/FileSearchService.cs. Place FileIndexBuilder near MaxDepth and FileSearchService near its MaxDepth constant.
Code tabs 8-9	Open: Clearspace/Services/TagService.cs near TagData, Load, Save, and Delete. Optionally also open FileIndexStore.cs and WindowsSearchService.cs for the last-minute buffer in the closing step.
Script and timing	Keep this PDF open beside the recording window. Silence notifications, start a 30-minute timer, and give each code screen a short pause before you begin speaking.

When every item is ready, start at 0:00 on the next page. Do not worry if a screen takes a few seconds to open - narrate the transition sentence while you move to it.

The 30-minute recording, in order

Follow the clock, not perfection. If you reach a time marker early, slowly scroll the code and briefly restate the point before advancing. Do not add features or claim that planned work is already completed.

0:00-1:30 - Start with the running application

SHOW	Clearspace open on a normal folder. Do not show code yet.
THEN SAY	Hello. This is my code review for Clearspace, a Windows desktop file manager. I selected this artifact because it combines a user-facing desktop application, performance-sensitive search, and persistent user metadata. In this review I will explain the existing application first. Then, for each required category, I will identify a specific limitation in the current code and explain the enhancement I plan to make.
PACE	Slowly point to the sidebar, file list, search box, and status area. Spend about 20 seconds letting the viewer see the application before moving on.
NEXT SAY	Now that I have introduced the artifact, I will briefly demonstrate the everyday file-management experience before reviewing the code behind it.

1:30-3:30 - Demonstrate basic user functionality

SHOW	Click through a folder, select an item, and use Back or the sidebar to move somewhere else.
THEN SAY	This first view shows the user problem Clearspace is solving. A user can browse folders, navigate with the sidebar and history controls, select files, and work with familiar file-manager behavior. The important design goal is that these common tasks remain responsive even when a folder has many items. That is why the project separates interface state from lower-level services that communicate with Windows and the file system.
PACE	Narrate each click. Pause briefly after navigation changes so the result is visible. This demonstration should take a full two minutes.
NEXT SAY	With the basic navigation visible, I will show two features that are especially relevant to the later technical review: search and tags.

3:30-5:00 - Demonstrate search and tags

SHOW	Type a simple search. If tags exist, select a file and show its tag menu or tag label.
THEN SAY	Clearspace also adds organization features beyond basic file browsing. Search can combine visible-folder results, the application's in-memory index, Windows Search, and a background crawl when necessary. Tags let a user group unrelated files without moving them on disk. I will return to both of these features because search drives the algorithms section and tags drive the database section.
PACE	Do not rush. Let search results appear, then clear the query. If the tag menu is unavailable, show the tag area and say it stores user-managed metadata.
NEXT SAY	Now that I have shown the application from a user's point of view, I will begin the first code-review category: software engineering and design.

5:00-7:00 - Explain the overall structure

SHOW	Open Clearspace/ARCHITECTURE.md. Keep the folder-layout section visible.
THEN SAY	I am opening the architecture document to explain the current organization. Clearspace has Native code for Windows calls, Models for application data, Services for reusable behavior, Commands for user actions, and ViewModels for user-interface state. This is a good starting separation because the window does not directly contain all of the file-system and Windows-specific work. I will now review whether the implementation maintains that separation as the application has grown.
PACE	Scroll through the folder-layout section, then pause on the text that describes Models, Services, Commands, and ViewModels.
NEXT SAY	This structure is the intended design. Next, I will show the main class where several of these responsibilities meet.

7:00-10:00 - Review the main coordinator

SHOW	Open Clearspace/ViewModels/MainViewModel.cs. Scroll to RunTreeSearchAsync near line 1085.
THEN SAY	This is the main view model, which coordinates what the user sees. It manages navigation, folder loading, search, sidebar entries, saved layout preferences, tags, and status information. That makes it central to the application, but it also makes it too broad. The RunTreeSearchAsync method is a useful example. It coordinates local search results, the Clearspace index, Windows Search, and the fallback directory crawl. This works, but a single class now has too many reasons to change. That is a maintainability and testing concern.
PACE	Scroll slowly around the method signature and the calls to the index, Windows Search, and FileSearchService. Give the viewer time to see the orchestration.
NEXT SAY	The view model coordinates the work, but commands carry out the user's actions. I will show that next.

10:00-12:30 - Review commands and file-operation feedback

SHOW	Open the Clearspace/Commands folder, then Clearspace/Services/FileOperationService.cs.
THEN SAY	The Commands folder is a positive design choice because user actions are represented separately from the interface. Menu items and keyboard shortcuts do not need to own all of their behavior directly. FileOperationService performs the Windows shell actions for copy, move, rename, and delete. The limitation is that these methods return only a Boolean result. If an operation returns false, the application cannot clearly tell the user whether it was cancelled, blocked by permissions, or failed for another reason. My software-engineering enhancement will use a richer result object and focused service interfaces so those outcomes can be handled clearly and tested.
PACE	Point to Copy, Move, Rename, Delete, and the Boolean Run method. Take 15 seconds to contrast the clean command separation with the limited error detail.
NEXT SAY	These two files show both the strengths and the limitations of the current design. I will now summarize the enhancement I plan for this category.

12:30-15:00 - Finish the software-engineering category

SHOW	Return to MainViewModel.cs or leave FileOperationService visible.
THEN SAY	To summarize the first category, Clearspace already has useful separation between the user interface, commands, and services. The improvement I plan is not to discard that design; it is to strengthen it. I will split the oversized main view model into focused navigation, search-coordination, and sidebar responsibilities. I will introduce clearer file-operation results and automated tests for search parsing, tag behavior, indexing edge cases, and user-visible failures. This enhancement demonstrates maintainability, testing, clear communication, and safer handling of user files.
PACE	Pause and explicitly say: "That concludes my software engineering and design review." This transition earns time and keeps the categories clear.
NEXT SAY	I have completed the software engineering and design category. I will now move to the algorithms and data-structures category, beginning with the search experience the user sees.

15:00-16:30 - Transition back to search

SHOW	Return to Clearspace and run another simple search.
THEN SAY	For the algorithms and data-structures category, I am returning to the search feature. A user expects search to respond quickly even when a drive contains many files. Clearspace therefore does not depend on just one technique. It can use the visible folder, an in-memory index, Windows Search, and a background crawl. The next files show how that indexing strategy works and what limitations I found.
PACE	Let the results appear. Explain that the application tries to return useful results quickly before moving to the code.
NEXT SAY	The app uses several search layers. The next file shows the in-memory data structure that makes repeated searches fast.

16:30-19:30 - Explain the in-memory index

SHOW	Open Clearspace/Services/FileIndex.cs. Show IndexEntry, NameOffset, ParentIndex, and Search.
THEN SAY	This file contains the core data structure for indexed search. An IndexEntry stores size, timestamps, attributes, a name offset, and a parent index. File names are stored in a shared character pool rather than as a separate object for every item. This is a deliberate memory tradeoff for large file collections. The ParentIndex lets Clearspace reconstruct a full path only for a result it needs to display. I also want to highlight the folded name pool. Names are converted to a consistent lower-case representation once, which avoids repeating that conversion every time a user changes a search query. The Search method then scans chunks in parallel and applies the user's filters.
PACE	Point to each field as you name it. Then scroll to Search and pause on Parallel.For. Spend at least 30 seconds explaining the memory-versus-speed tradeoff in your own words.
NEXT SAY	The data structure is only useful if it can be built safely. Next, I will show how Clearspace walks the file system to build this index.

19:30-22:00 - Review how the index is built

SHOW	Open Clearspace/Services/FileIndexBuilder.cs. Keep MaxDepth and Build visible.
THEN SAY	This builder creates the index by walking directories in the background. It uses a stack to record directories that still need to be scanned. It also avoids following reparse points, which prevents symbolic links and junctions from creating cycles. This is a sound defensive choice. However, the key limitation is visible near the top of the file: MaxDepth is set to 32. A file deeper than that limit may never enter the index. That creates a correctness problem because the application describes its search as comprehensive.
PACE	Point to MaxDepth, then scroll to the pending Stack and the depth check. Pause after saying "correctness problem" so the reviewer sees the exact condition.
NEXT SAY	That depth limit is important, so I will compare it with the separate fallback-search algorithm.

22:00-24:00 - Compare the fallback algorithm

SHOW	Open Clearspace/Services/FileSearchService.cs. Show MaxDepth at line 14 and the worker loop.
THEN SAY	This is the fallback search path for locations that are not fully covered by the in-memory index. It uses concurrent workers and publishes results in batches, which is good for a responsive interface. But it has a second depth limit, and this one is set to 24. That means the fallback can be less complete than the main index. My enhancement will make traversal depth configurable or remove the arbitrary cap, then test both search paths with the same deep directory structures, cancellation conditions, rename operations, and deletion cases. The goal is that a fallback remains correct, not merely fast.
PACE	Slowly compare the MaxDepth constant in this file with the earlier one. Point out the queue, worker tasks, and batch publishing.
NEXT SAY	With both search paths reviewed, I can now describe the specific algorithm and data-structure enhancement I plan to implement.

24:00-26:00 - State the algorithms enhancement

SHOW	Return to FileIndex.cs and show ParentIndex again.
THEN SAY	My planned algorithms enhancement is recursive folder-size aggregation. The existing index already stores each file's size and its parent index. After an index build, I can use those parent relationships in a bottom-up pass to calculate the total size of each folder and its descendants. This lets Clearspace display folder sizes without walking the disk again every time the user asks. I will measure the memory and time cost, add correctness tests, and keep index health per volume rather than treating one watcher failure as a failure for every drive. That demonstrates the tradeoffs involved in data structures, concurrency, and performance.
PACE	Pause on ParentIndex while explaining the bottom-up pass. Then clearly state: "That concludes my algorithms and data-structures review."
NEXT SAY	That completes the algorithms and data-structures category. I will now return to the application's tag feature to begin the database review.

26:00-27:30 - Return to tags in the running app

SHOW	Return to Clearspace. Select an item and show a tag, tag menu, or tag-related search filter.
THEN SAY	For the database category, I am returning to tags. Tags are user-managed metadata. A user can apply more than one tag to a file, and the same tag can apply to many files. That is a many-to-many relationship. The current feature is useful and simple for a small amount of data, but the storage design becomes important as the user assigns more tags over time.
PACE	Spend 20 seconds showing a tag or explaining the tag UI. Do not worry if you cannot create a new tag during the recording; showing the existing menu or metadata is enough.
NEXT SAY	The user-facing tag feature is simple, but the next file shows how its data is currently stored and why that storage can be improved.

27:30-29:00 - Review the current tag persistence

SHOW	Open Clearspace/Services/TagService.cs. Show TagData near line 43, Load, Save, and Delete.
THEN SAY	This service shows how tags are stored today. TagData contains a list of tag definitions and a dictionary that maps a file path to a list of tag IDs. This works, but every update rewrites the full JSON document. When a tag is deleted, the application manually scans assignments to remove orphaned tag IDs. That means the application code is responsible for relationship cleanup, and there is no transaction protecting a multi-step update. This is where a relational database can improve data integrity and query performance.
PACE	Point to Definitions and Assignments first. Then scroll to Save and Delete. Give the viewer a moment to see the full-document write and manual cleanup loop.
NEXT SAY	I have now identified the data-integrity and scalability limits of the JSON approach. I will close by explaining the SQLite enhancement that addresses those limits.

29:00-30:00 - State the database enhancement and close

SHOW	Leave TagService.cs open. You may also briefly show FileIndexStore.cs or WindowsSearchService.cs if time permits.
THEN SAY	My planned database enhancement is a local SQLite database for tag metadata. I will use a Tags table and a FileTags table to model the many-to-many relationship. I will enable foreign keys, add an index for tag lookups, use parameterized queries, and perform updates in transactions. I will also migrate existing JSON data safely and keep a backup until migration succeeds. Clearspace already has a strong foundation, and these three enhancements will make it easier to maintain, more accurate when searching, and more reliable when storing user data. Thank you for reviewing my code-review plan.
PACE	End cleanly at 30 minutes. If you are a little early, briefly show FileIndexStore.cs and say that the SQLite work improves tags only; it does not replace the high-performance file index or Windows Search.
NEXT SAY	This concludes my code review. The planned enhancements build on the current application rather than replacing it, and together they improve maintainability, search correctness, and data reliability.